Chess Concept Library

McGrath et al. (2022), “Acquisition of Chess Knowledge in AlphaZero”, PNAS — Tables S1–S3. 207 concepts: 93 Stockfish 8 + 114 custom.

Part 1 — Stockfish 8 evaluation concepts (Table S1) — 93 concepts

Concept name	Description
`material_t_mg`	Material score. Every piece on the board has a predefined value that changes with the phase of the game, and each piece is valued in isolation, independently of the other pieces (contrast with `imbalance`). *[total/both sides combined; middlegame value]*
`material_t_eg`	Material score. Every piece on the board has a predefined value that changes with the phase of the game, and each piece is valued in isolation, independently of the other pieces (contrast with `imbalance`). *[total/both sides combined; endgame value]*
`material_t_ph`	Material score. Every piece on the board has a predefined value that changes with the phase of the game, and each piece is valued in isolation, independently of the other pieces (contrast with `imbalance`). *[total/both sides combined; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`imbalance_t_mg`	Second-order material score: the value of each piece is adjusted with respect to all other pieces on the board, favouring or penalising particular piece-count combinations (e.g. a bishop pair is preferred over bishop plus knight). Computed as a quadratic form of the piece counts with a preset weight matrix. *[total/both sides combined; middlegame value]*
`imbalance_t_eg`	Second-order material score: the value of each piece is adjusted with respect to all other pieces on the board, favouring or penalising particular piece-count combinations (e.g. a bishop pair is preferred over bishop plus knight). Computed as a quadratic form of the piece counts with a preset weight matrix. *[total/both sides combined; endgame value]*
`imbalance_t_ph`	Second-order material score: the value of each piece is adjusted with respect to all other pieces on the board, favouring or penalising particular piece-count combinations (e.g. a bishop pair is preferred over bishop plus knight). Computed as a quadratic form of the piece counts with a preset weight matrix. *[total/both sides combined; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`pawns_t_mg`	Pawn-structure evaluation: isolated, doubled, connected, backward, blocked, weak pawns and similar features. *[total/both sides combined; middlegame value]*
`pawns_t_eg`	Pawn-structure evaluation: isolated, doubled, connected, backward, blocked, weak pawns and similar features. *[total/both sides combined; endgame value]*
`pawns_t_ph`	Pawn-structure evaluation: isolated, doubled, connected, backward, blocked, weak pawns and similar features. *[total/both sides combined; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`knights_mine_mg`	Knight evaluation; e.g. extra points for knights occupying outposts protected by pawns. *[for the side to move; middlegame value]*
`knights_mine_eg`	Knight evaluation; e.g. extra points for knights occupying outposts protected by pawns. *[for the side to move; endgame value]*
`knights_mine_ph`	Knight evaluation; e.g. extra points for knights occupying outposts protected by pawns. *[for the side to move; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`knights_opponent_mg`	Knight evaluation; e.g. extra points for knights occupying outposts protected by pawns. *[for the opposing side; middlegame value]*
`knights_opponent_eg`	Knight evaluation; e.g. extra points for knights occupying outposts protected by pawns. *[for the opposing side; endgame value]*
`knights_opponent_ph`	Knight evaluation; e.g. extra points for knights occupying outposts protected by pawns. *[for the opposing side; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`knights_t_mg`	Knight evaluation; e.g. extra points for knights occupying outposts protected by pawns. *[total/both sides combined; middlegame value]*

Part 1 — Stockfish 8 evaluation concepts (Table S1) — 93 concepts	
Concept name	**Description**
`knights_t_eg`	Knight evaluation; e.g. extra points for knights occupying outposts protected by pawns. *[total/both sides combined; endgame value]*
`knights_t_ph`	Knight evaluation; e.g. extra points for knights occupying outposts protected by pawns. *[total/both sides combined; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`bishop_mine_mg`	Bishop evaluation; e.g. bishops standing on squares of the same colour as their own pawns are penalised. (The paper's probe labels use the singular `bishop`.) *[for the side to move; middlegame value]*
`bishop_mine_eg`	Bishop evaluation; e.g. bishops standing on squares of the same colour as their own pawns are penalised. (The paper's probe labels use the singular `bishop`.) *[for the side to move; endgame value]*
`bishop_mine_ph`	Bishop evaluation; e.g. bishops standing on squares of the same colour as their own pawns are penalised. (The paper's probe labels use the singular `bishop`.) *[for the side to move; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`bishop_opponent_mg`	Bishop evaluation; e.g. bishops standing on squares of the same colour as their own pawns are penalised. (The paper's probe labels use the singular `bishop`.) *[for the opposing side; middlegame value]*
`bishop_opponent_eg`	Bishop evaluation; e.g. bishops standing on squares of the same colour as their own pawns are penalised. (The paper's probe labels use the singular `bishop`.) *[for the opposing side; endgame value]*
`bishop_opponent_ph`	Bishop evaluation; e.g. bishops standing on squares of the same colour as their own pawns are penalised. (The paper's probe labels use the singular `bishop`.) *[for the opposing side; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`bishop_t_mg`	Bishop evaluation; e.g. bishops standing on squares of the same colour as their own pawns are penalised. (The paper's probe labels use the singular `bishop`.) *[total/both sides combined; middlegame value]*
`bishop_t_eg`	Bishop evaluation; e.g. bishops standing on squares of the same colour as their own pawns are penalised. (The paper's probe labels use the singular `bishop`.) *[total/both sides combined; endgame value]*
`bishop_t_ph`	Bishop evaluation; e.g. bishops standing on squares of the same colour as their own pawns are penalised. (The paper's probe labels use the singular `bishop`.) *[total/both sides combined; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`rooks_mine_mg`	Rook evaluation; e.g. rooks occupying open or semi-open files are valued more highly. *[for the side to move; middlegame value]*
`rooks_mine_eg`	Rook evaluation; e.g. rooks occupying open or semi-open files are valued more highly. *[for the side to move; endgame value]*
`rooks_mine_ph`	Rook evaluation; e.g. rooks occupying open or semi-open files are valued more highly. *[for the side to move; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`rooks_opponent_mg`	Rook evaluation; e.g. rooks occupying open or semi-open files are valued more highly. *[for the opposing side; middlegame value]*
`rooks_opponent_eg`	Rook evaluation; e.g. rooks occupying open or semi-open files are valued more highly. *[for the opposing side; endgame value]*
`rooks_opponent_ph`	Rook evaluation; e.g. rooks occupying open or semi-open files are valued more highly. *[for the opposing side; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`rooks_t_mg`	Rook evaluation; e.g. rooks occupying open or semi-open files are valued more highly. *[total/both sides combined; middlegame value]*

Part 1 — Stockfish 8 evaluation concepts (Table S1) — 93 concepts	
Concept name	**Description**
`rooks_t_eg`	Rook evaluation; e.g. rooks occupying open or semi-open files are valued more highly. *[total/both sides combined; endgame value]*
`rooks_t_ph`	Rook evaluation; e.g. rooks occupying open or semi-open files are valued more highly. *[total/both sides combined; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`queens_mine_mg`	Queen evaluation; e.g. queens that are subject to a relative pin or a discovered attack are penalised. *[for the side to move; middlegame value]*
`queens_mine_eg`	Queen evaluation; e.g. queens that are subject to a relative pin or a discovered attack are penalised. *[for the side to move; endgame value]*
`queens_mine_ph`	Queen evaluation; e.g. queens that are subject to a relative pin or a discovered attack are penalised. *[for the side to move; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`queens_opponent_mg`	Queen evaluation; e.g. queens that are subject to a relative pin or a discovered attack are penalised. *[for the opposing side; middlegame value]*
`queens_opponent_eg`	Queen evaluation; e.g. queens that are subject to a relative pin or a discovered attack are penalised. *[for the opposing side; endgame value]*
`queens_opponent_ph`	Queen evaluation; e.g. queens that are subject to a relative pin or a discovered attack are penalised. *[for the opposing side; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`queens_t_mg`	Queen evaluation; e.g. queens that are subject to a relative pin or a discovered attack are penalised. *[total/both sides combined; middlegame value]*
`queens_t_eg`	Queen evaluation; e.g. queens that are subject to a relative pin or a discovered attack are penalised. *[total/both sides combined; endgame value]*
`queens_t_ph`	Queen evaluation; e.g. queens that are subject to a relative pin or a discovered attack are penalised. *[total/both sides combined; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`mobility_mine_mg`	Piece-mobility score, depending on the number of squares attacked by the side's pieces. *[for the side to move; middlegame value]*
`mobility_mine_eg`	Piece-mobility score, depending on the number of squares attacked by the side's pieces. *[for the side to move; endgame value]*
`mobility_mine_ph`	Piece-mobility score, depending on the number of squares attacked by the side's pieces. *[for the side to move; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`mobility_opponent_mg`	Piece-mobility score, depending on the number of squares attacked by the side's pieces. *[for the opposing side; middlegame value]*
`mobility_opponent_eg`	Piece-mobility score, depending on the number of squares attacked by the side's pieces. *[for the opposing side; endgame value]*
`mobility_opponent_ph`	Piece-mobility score, depending on the number of squares attacked by the side's pieces. *[for the opposing side; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`mobility_t_mg`	Piece-mobility score, depending on the number of squares attacked by the side's pieces. *[total/both sides combined; middlegame value]*
`mobility_t_eg`	Piece-mobility score, depending on the number of squares attacked by the side's pieces. *[total/both sides combined; endgame value]*
`mobility_t_ph`	Piece-mobility score, depending on the number of squares attacked by the side's pieces. *[total/both sides combined; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`king_safety_mine_mg`	Complex king-safety term: number and type of pieces attacking the squares around the king, pawn-shelter strength, number of pawns around the king, penalties for a king on a pawnless flank, and similar features. *[for the side to move; middlegame value]*

Part 1 — Stockfish 8 evaluation concepts (Table S1) — 93 concepts

Concept name	Description
`king_safety_mine_eg`	Complex king-safety term: number and type of pieces attacking the squares around the king, pawn-shelter strength, number of pawns around the king, penalties for a king on a pawnless flank, and similar features. *[for the side to move; endgame value]*
`king_safety_mine_ph`	Complex king-safety term: number and type of pieces attacking the squares around the king, pawn-shelter strength, number of pawns around the king, penalties for a king on a pawnless flank, and similar features. *[for the side to move; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`king_safety_opponent_mg`	Complex king-safety term: number and type of pieces attacking the squares around the king, pawn-shelter strength, number of pawns around the king, penalties for a king on a pawnless flank, and similar features. *[for the opposing side; middlegame value]*
`king_safety_opponent_eg`	Complex king-safety term: number and type of pieces attacking the squares around the king, pawn-shelter strength, number of pawns around the king, penalties for a king on a pawnless flank, and similar features. *[for the opposing side; endgame value]*
`king_safety_opponent_ph`	Complex king-safety term: number and type of pieces attacking the squares around the king, pawn-shelter strength, number of pawns around the king, penalties for a king on a pawnless flank, and similar features. *[for the opposing side; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`king_safety_t_mg`	Complex king-safety term: number and type of pieces attacking the squares around the king, pawn-shelter strength, number of pawns around the king, penalties for a king on a pawnless flank, and similar features. *[total/both sides combined; middlegame value]*
`king_safety_t_eg`	Complex king-safety term: number and type of pieces attacking the squares around the king, pawn-shelter strength, number of pawns around the king, penalties for a king on a pawnless flank, and similar features. *[total/both sides combined; endgame value]*
`king_safety_t_ph`	Complex king-safety term: number and type of pieces attacking the squares around the king, pawn-shelter strength, number of pawns around the king, penalties for a king on a pawnless flank, and similar features. *[total/both sides combined; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`threats_mine_mg`	Threats against pieces: whether a pawn can safely advance and attack a higher-value enemy piece, hanging pieces, possible x-ray attacks by rooks, and similar features. *[for the side to move; middlegame value]*
`threats_mine_eg`	Threats against pieces: whether a pawn can safely advance and attack a higher-value enemy piece, hanging pieces, possible x-ray attacks by rooks, and similar features. *[for the side to move; endgame value]*
`threats_mine_ph`	Threats against pieces: whether a pawn can safely advance and attack a higher-value enemy piece, hanging pieces, possible x-ray attacks by rooks, and similar features. *[for the side to move; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`threats_opponent_mg`	Threats against pieces: whether a pawn can safely advance and attack a higher-value enemy piece, hanging pieces, possible x-ray attacks by rooks, and similar features. *[for the opposing side; middlegame value]*
`threats_opponent_eg`	Threats against pieces: whether a pawn can safely advance and attack a higher-value enemy piece, hanging pieces, possible x-ray attacks by rooks, and similar features. *[for the opposing side; endgame value]*
`threats_opponent_ph`	Threats against pieces: whether a pawn can safely advance and attack a higher-value enemy piece, hanging pieces, possible x-ray attacks by rooks, and similar features. *[for the opposing side; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`threats_t_mg`	Threats against pieces: whether a pawn can safely advance and attack a higher-value enemy piece, hanging pieces, possible x-ray attacks by rooks, and similar features. *[total/both sides combined; middlegame value]*
`threats_t_eg`	Threats against pieces: whether a pawn can safely advance and attack a higher-value enemy piece, hanging pieces, possible x-ray attacks by rooks, and similar features. *[total/both sides combined; endgame value]*

Part 1 — Stockfish 8 evaluation concepts (Table S1) — 93 concepts	
Concept name	**Description**
`threats_t_ph`	Threats against pieces: whether a pawn can safely advance and attack a higher-value enemy piece, hanging pieces, possible x-ray attacks by rooks, and similar features. *[total/both sides combined; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`passed_pawns_mine_mg`	Bonuses for passed pawns; the closer a pawn is to the promotion rank, the larger the bonus. *[for the side to move; middlegame value]*
`passed_pawns_mine_eg`	Bonuses for passed pawns; the closer a pawn is to the promotion rank, the larger the bonus. *[for the side to move; endgame value]*
`passed_pawns_mine_ph`	Bonuses for passed pawns; the closer a pawn is to the promotion rank, the larger the bonus. *[for the side to move; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`passed_pawns_opponent_mg`	Bonuses for passed pawns; the closer a pawn is to the promotion rank, the larger the bonus. *[for the opposing side; middlegame value]*
`passed_pawns_opponent_eg`	Bonuses for passed pawns; the closer a pawn is to the promotion rank, the larger the bonus. *[for the opposing side; endgame value]*
`passed_pawns_opponent_ph`	Bonuses for passed pawns; the closer a pawn is to the promotion rank, the larger the bonus. *[for the opposing side; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`passed_pawns_t_mg`	Bonuses for passed pawns; the closer a pawn is to the promotion rank, the larger the bonus. *[total/both sides combined; middlegame value]*
`passed_pawns_t_eg`	Bonuses for passed pawns; the closer a pawn is to the promotion rank, the larger the bonus. *[total/both sides combined; endgame value]*
`passed_pawns_t_ph`	Bonuses for passed pawns; the closer a pawn is to the promotion rank, the larger the bonus. *[total/both sides combined; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`space_mine_mg`	Space evaluation: number of safe squares available for minor pieces on the central four files, on ranks 2 to 4. *[for the side to move; middlegame value]*
`space_mine_eg`	Space evaluation: number of safe squares available for minor pieces on the central four files, on ranks 2 to 4. *[for the side to move; endgame value]*
`space_mine_ph`	Space evaluation: number of safe squares available for minor pieces on the central four files, on ranks 2 to 4. *[for the side to move; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`space_opponent_mg`	Space evaluation: number of safe squares available for minor pieces on the central four files, on ranks 2 to 4. *[for the opposing side; middlegame value]*
`space_opponent_eg`	Space evaluation: number of safe squares available for minor pieces on the central four files, on ranks 2 to 4. *[for the opposing side; endgame value]*
`space_opponent_ph`	Space evaluation: number of safe squares available for minor pieces on the central four files, on ranks 2 to 4. *[for the opposing side; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`space_t_mg`	Space evaluation: number of safe squares available for minor pieces on the central four files, on ranks 2 to 4. *[total/both sides combined; middlegame value]*
`space_t_eg`	Space evaluation: number of safe squares available for minor pieces on the central four files, on ranks 2 to 4. *[total/both sides combined; endgame value]*
`space_t_ph`	Space evaluation: number of safe squares available for minor pieces on the central four files, on ranks 2 to 4. *[total/both sides combined; phase-weighted value (middlegame/endgame mix based on remaining material)]*
`total_t_mg`	The total evaluation of the position, encapsulating all of the concepts above. *[total/both sides combined; middlegame value]*
`total_t_eg`	The total evaluation of the position, encapsulating all of the concepts above. *[total/both sides combined; endgame value]*

Part 1 — Stockfish 8 evaluation concepts (Table S1) — 93 concepts

Concept name	Description
`total_t_ph`	The total evaluation of the position, encapsulating all of the concepts above. *[total/both sides combined; phase-weighted value (middlegame/endgame mix based on remaining material)]*

Part 2 — Custom concepts (Table S2) — 78 concepts

Concept name	Description
`pawn_fork_mine`	True if the side has a pawn that attacks two enemy pieces of higher value (knight, bishop, rook, queen or king) and the pawn is not pinned. *[for the side to move]*
`pawn_fork_opponent`	True if the side has a pawn that attacks two enemy pieces of higher value (knight, bishop, rook, queen or king) and the pawn is not pinned. *[for the opposing side]*
`knight_fork_mine`	True if the side has a knight that attacks two enemy pieces of higher value (rook, queen or king) and the knight is not pinned. *[for the side to move]*
`knight_fork_opponent`	True if the side has a knight that attacks two enemy pieces of higher value (rook, queen or king) and the knight is not pinned. *[for the opposing side]*
`bishop_fork_mine`	True if the side has a bishop that attacks two enemy pieces of higher value (rook, queen or king) and the bishop is not pinned. *[for the side to move]*
`bishop_fork_opponent`	True if the side has a bishop that attacks two enemy pieces of higher value (rook, queen or king) and the bishop is not pinned. *[for the opposing side]*
`rook_fork_mine`	True if the side has a rook that attacks two enemy pieces of higher value (queen or king) and the rook is not pinned. *[for the side to move]*
`rook_fork_opponent`	True if the side has a rook that attacks two enemy pieces of higher value (queen or king) and the rook is not pinned. *[for the opposing side]*
`has_pinned_pawn_mine`	True if the side has a pawn that is pinned to its own king. *[for the side to move]*
`has_pinned_pawn_opponent`	True if the side has a pawn that is pinned to its own king. *[for the opposing side]*
`has_pinned_knight_mine`	True if the side has a knight that is pinned to its own king. *[for the side to move]*
`has_pinned_knight_opponent`	True if the side has a knight that is pinned to its own king. *[for the opposing side]*
`has_pinned_bishop_mine`	True if the side has a bishop that is pinned to its own king. *[for the side to move]*
`has_pinned_bishop_opponent`	True if the side has a bishop that is pinned to its own king. *[for the opposing side]*
`has_pinned_rook_mine`	True if the side has a rook that is pinned to its own king. *[for the side to move]*
`has_pinned_rook_opponent`	True if the side has a rook that is pinned to its own king. *[for the opposing side]*
`has_pinned_queen_mine`	True if the side has a queen that is pinned to its own king. *[for the side to move]*
`has_pinned_queen_opponent`	True if the side has a queen that is pinned to its own king. *[for the opposing side]*
`material_mine`	Simple material count: (#pawns) + 3*(#knights) + 3*(#bishops) + 5*(#rooks) + 9*(#queens). Distinct from the Stockfish `material` concept in Part 1. *[for the side to move]*
`material_opponent`	Simple material count: (#pawns) + 3*(#knights) + 3*(#bishops) + 5*(#rooks) + 9*(#queens). Distinct from the Stockfish `material` concept in Part 1. *[for the opposing side]*
`material_diff`	Simple material count: (#pawns) + 3*(#knights) + 3*(#bishops) + 5*(#rooks) + 9*(#queens). Distinct from the Stockfish `material` concept in Part 1. *[difference between the side to move and the opponent]*
`num_pieces_mine`	Number of pieces that the side has. *[for the side to move]*
`num_pieces_opponent`	Number of pieces that the side has. *[for the opposing side]*
`num_pieces_diff`	Number of pieces that the side has. *[difference between the side to move and the opponent]*

Part 2 — Custom concepts (Table S2) — 78 concepts

Concept name	Description
`in_check`	True if the side to move is in check.
`has_bishop_pair_mine`	True if the side has a pair of bishops. *[for the side to move]*
`has_bishop_pair_opponent`	True if the side has a pair of bishops. *[for the opposing side]*
`has_connected_rooks_mine`	True if the side has connected rooks. *[for the side to move]*
`has_connected_rooks_opponent`	True if the side has connected rooks. *[for the opposing side]*
`has_control_of_open_file_mine`	True if the side controls an open file (with its rooks and/or queen). *[for the side to move]*
`has_control_of_open_file_opponent`	True if the side controls an open file (with its rooks and/or queen). *[for the opposing side]*
`has_mate_threat`	True if the opponent could mate the side to move in a single move if the turn were passed to the opponent (i.e. the mover is under a mate-in-one threat).
`has_check_move_mine`	True if the side has a move that gives check to the enemy king. *[for the side to move]*
`has_check_move_opponent`	True if the side has a move that gives check to the enemy king. *[for the opposing side]*
`can_capture_queen_mine`	True if the side can capture the opponent's queen. *[for the side to move]*
`can_capture_queen_opponent`	True if the side can capture the opponent's queen. *[for the opposing side]*
`num_king_attacked_squares_mine`	Number of squares around the enemy king that the given side attacks; occupied squares can be included. *[for the side to move]*
`num_king_attacked_squares_opponent`	Number of squares around the enemy king that the given side attacks; occupied squares can be included. *[for the opposing side]*
`num_king_attacked_squares_diff`	Number of squares around the enemy king that the given side attacks; occupied squares can be included. *[difference between the side to move and the opponent]*
`has_contested_open_file`	True if an open file is occupied simultaneously by a rook and/or queen of both colours.
`has_right_bc_ha_promotion_mine`	True if the side has (1) a passed pawn on the a- or h-file and (2) a bishop whose square colour matches the promotion square of that pawn (the "right-coloured bishop" for a rook-pawn promotion). *[for the side to move]*
`has_right_bc_ha_promotion_opponent`	True if the side has (1) a passed pawn on the a- or h-file and (2) a bishop whose square colour matches the promotion square of that pawn (the "right-coloured bishop" for a rook-pawn promotion). *[for the opposing side]*
`num_scb_pawns_same_side_mine`	Number of the side's own pawns that occupy squares of the same colour as its own bishop. Only applicable when the side has a single bishop. *[for the side to move]*
`num_scb_pawns_same_side_opponent`	Number of the side's own pawns that occupy squares of the same colour as its own bishop. Only applicable when the side has a single bishop. *[for the opposing side]*
`num_scb_pawns_same_side_diff`	Number of the side's own pawns that occupy squares of the same colour as its own bishop. Only applicable when the side has a single bishop. *[difference between the side to move and the opponent]*
`num_ocb_pawns_same_side_mine`	Number of the side's own pawns that occupy squares of the opposite colour to its own bishop. Only applicable when the side has a single bishop. *[for the side to move]*
`num_ocb_pawns_same_side_opponent`	Number of the side's own pawns that occupy squares of the opposite colour to its own bishop. Only applicable when the side has a single bishop. *[for the opposing side]*
`num_ocb_pawns_same_side_diff`	Number of the side's own pawns that occupy squares of the opposite colour to its own bishop. Only applicable when the side has a single bishop. *[difference between the side to move and the opponent]*
`num_scb_pawns_other_side_mine`	Number of the opponent's pawns that occupy squares of the same colour as the side's own bishop. Only applicable when the side has a single bishop. *[for the side to move]*

Part 2 — Custom concepts (Table S2) — 78 concepts	
Concept name	**Description**
`num_scb_pawns_other_side_opponent`	Number of the opponent's pawns that occupy squares of the same colour as the side's own bishop. Only applicable when the side has a single bishop. *[for the opposing side]*
`num_scb_pawns_other_side_diff`	Number of the opponent's pawns that occupy squares of the same colour as the side's own bishop. Only applicable when the side has a single bishop. *[difference between the side to move and the opponent]*
`num_ocb_pawns_other_side_mine`	Number of the opponent's pawns that occupy squares of the opposite colour to the side's own bishop. Only applicable when the side has a single bishop. *[for the side to move]*
`num_ocb_pawns_other_side_opponent`	Number of the opponent's pawns that occupy squares of the opposite colour to the side's own bishop. Only applicable when the side has a single bishop. *[for the opposing side]*
`num_ocb_pawns_other_side_diff`	Number of the opponent's pawns that occupy squares of the opposite colour to the side's own bishop. Only applicable when the side has a single bishop. *[difference between the side to move and the opponent]*
`capture_possible_on_d1_mine`	True if the side can capture a piece on the given square. Squares are named as if the side were playing White. *[for the side to move]*
`capture_possible_on_d1_opponent`	True if the side can capture a piece on the given square. Squares are named as if the side were playing White. *[for the opposing side]*
`capture_possible_on_d2_mine`	True if the side can capture a piece on the given square. Squares are named as if the side were playing White. *[for the side to move]*
`capture_possible_on_d2_opponent`	True if the side can capture a piece on the given square. Squares are named as if the side were playing White. *[for the opposing side]*
`capture_possible_on_d3_mine`	True if the side can capture a piece on the given square. Squares are named as if the side were playing White. *[for the side to move]*
`capture_possible_on_d3_opponent`	True if the side can capture a piece on the given square. Squares are named as if the side were playing White. *[for the opposing side]*
`capture_possible_on_e1_mine`	True if the side can capture a piece on the given square. Squares are named as if the side were playing White. *[for the side to move]*
`capture_possible_on_e1_opponent`	True if the side can capture a piece on the given square. Squares are named as if the side were playing White. *[for the opposing side]*
`capture_possible_on_e2_mine`	True if the side can capture a piece on the given square. Squares are named as if the side were playing White. *[for the side to move]*
`capture_possible_on_e2_opponent`	True if the side can capture a piece on the given square. Squares are named as if the side were playing White. *[for the opposing side]*
`capture_possible_on_e3_mine`	True if the side can capture a piece on the given square. Squares are named as if the side were playing White. *[for the side to move]*
`capture_possible_on_e3_opponent`	True if the side can capture a piece on the given square. Squares are named as if the side were playing White. *[for the opposing side]*
`capture_possible_on_g5_mine`	True if the side can capture a piece on the given square. Squares are named as if the side were playing White. *[for the side to move]*
`capture_possible_on_g5_opponent`	True if the side can capture a piece on the given square. Squares are named as if the side were playing White. *[for the opposing side]*
`capture_possible_on_b5_mine`	True if the side can capture a piece on the given square. Squares are named as if the side were playing White. *[for the side to move]*
`capture_possible_on_b5_opponent`	True if the side can capture a piece on the given square. Squares are named as if the side were playing White. *[for the opposing side]*
`capture_happens_next_move_on_d1`	True if a capture of a piece on the given square actually happened, according to the game record - a game-trajectory label rather than a static board property. Squares are named as if the side were playing White.

Part 2 — Custom concepts (Table S2) — 78 concepts

Concept name	Description
`capture_happens_next_move_on_d2`	True if a capture of a piece on the given square actually happened, according to the game record - a game-trajectory label rather than a static board property. Squares are named as if the side were playing White.
`capture_happens_next_move_on_d3`	True if a capture of a piece on the given square actually happened, according to the game record - a game-trajectory label rather than a static board property. Squares are named as if the side were playing White.
`capture_happens_next_move_on_e1`	True if a capture of a piece on the given square actually happened, according to the game record - a game-trajectory label rather than a static board property. Squares are named as if the side were playing White.
`capture_happens_next_move_on_e2`	True if a capture of a piece on the given square actually happened, according to the game record - a game-trajectory label rather than a static board property. Squares are named as if the side were playing White.
`capture_happens_next_move_on_e3`	True if a capture of a piece on the given square actually happened, according to the game record - a game-trajectory label rather than a static board property. Squares are named as if the side were playing White.
`capture_happens_next_move_on_g5`	True if a capture of a piece on the given square actually happened, according to the game record - a game-trajectory label rather than a static board property. Squares are named as if the side were playing White.
`capture_happens_next_move_on_b5`	True if a capture of a piece on the given square actually happened, according to the game record - a game-trajectory label rather than a static board property. Squares are named as if the side were playing White.

Part 3 — Custom pawn-structure concepts (Table S3) — 36 concepts

Concept name	Description
`num_double_pawn_files_t`	Number of files that carry more than one pawn of the side's colour (files with doubled pawns). *[total/both sides combined]*
`num_double_pawn_files_mine`	Number of files that carry more than one pawn of the side's colour (files with doubled pawns). *[for the side to move]*
`num_double_pawn_files_opponent`	Number of files that carry more than one pawn of the side's colour (files with doubled pawns). *[for the opposing side]*
`num_double_pawn_files_diff`	Number of files that carry more than one pawn of the side's colour (files with doubled pawns). *[difference between the side to move and the opponent]*
`has_double_pawn_mine`	True if the side has doubled pawns, i.e. some file contains more than one of its pawns. *[for the side to move]*
`has_double_pawn_opponent`	True if the side has doubled pawns, i.e. some file contains more than one of its pawns. *[for the opposing side]*
`num_isolated_pawns_mine`	Number of the side's pawns that have no friendly pawns on the files to their left and right. *[for the side to move]*
`num_isolated_pawns_opponent`	Number of the side's pawns that have no friendly pawns on the files to their left and right. *[for the opposing side]*
`num_isolated_pawns_diff`	Number of the side's pawns that have no friendly pawns on the files to their left and right. *[difference between the side to move and the opponent]*
`has_isolated_pawn_mine`	True if the side has a pawn with no friendly pawns on the adjacent files. *[for the side to move]*
`has_isolated_pawn_opponent`	True if the side has a pawn with no friendly pawns on the adjacent files. *[for the opposing side]*
`has_pawn_on_7th_rank_mine`	True if the side has a pawn that has reached the 7th rank (from its own perspective; one step from promotion). *[for the side to move]*

Part 3 — Custom pawn-structure concepts (Table S3) — 36 concepts

Concept name	Description
has_pawn_on_7th_rank_opponent	True if the side has a pawn that has reached the 7th rank (from its own perspective; one step from promotion). *[for the opposing side]*
pawns_on_7th_rank_mine	Number of the side's pawns that have reached the 7th rank. *[for the side to move]*
pawns_on_7th_rank_opponent	Number of the side's pawns that have reached the 7th rank. *[for the opposing side]*
pawns_on_7th_rank_diff	Number of the side's pawns that have reached the 7th rank. *[difference between the side to move and the opponent]*
has_passed_pawn_mine	True if the side has a pawn with no opposing pawns able to prevent it from advancing to the eighth rank. *[for the side to move]*
has_passed_pawn_opponent	True if the side has a pawn with no opposing pawns able to prevent it from advancing to the eighth rank. *[for the opposing side]*
num_passed_pawns_mine	Number of passed pawns the side has. *[for the side to move]*
num_passed_pawns_opponent	Number of passed pawns the side has. *[for the opposing side]*
num_passed_pawns_diff	Number of passed pawns the side has. *[difference between the side to move and the opponent]*
has_protected_passed_pawn_mine	True if the side has a passed pawn that is protected by one of its own pawns. *[for the side to move]*
has_protected_passed_pawn_opponent	True if the side has a passed pawn that is protected by one of its own pawns. *[for the opposing side]*
num_protected_passed_pawns_mine	Number of protected passed pawns the side has. *[for the side to move]*
num_protected_passed_pawns_opponent	Number of protected passed pawns the side has. *[for the opposing side]*
num_protected_passed_pawns_diff	Number of protected passed pawns the side has. *[difference between the side to move and the opponent]*
num_pawn_islands_mine	Number of pawn islands (maximal groups of the side's pawns on consecutive files, separated by files without its pawns). *[for the side to move]*
num_pawn_islands_opponent	Number of pawn islands (maximal groups of the side's pawns on consecutive files, separated by files without its pawns). *[for the opposing side]*
num_pawn_islands_diff	Number of pawn islands (maximal groups of the side's pawns on consecutive files, separated by files without its pawns). *[difference between the side to move and the opponent]*
has_iqp_mine	True if the side has an isolated queen's pawn (an isolated pawn on the d-file). *[for the side to move]*
has_iqp_opponent	True if the side has an isolated queen's pawn (an isolated pawn on the d-file). *[for the opposing side]*
has_connected_passed_pawns_mine	True if the side has two or more passed pawns on adjacent files. *[for the side to move]*
has_connected_passed_pawns_opponent	True if the side has two or more passed pawns on adjacent files. *[for the opposing side]*
num_connected_passed_pawns_mine	Number of connected passed pawns the side has. *[for the side to move]*
num_connected_passed_pawns_opponent	Number of connected passed pawns the side has. *[for the opposing side]*
num_connected_passed_pawns_diff	Number of connected passed pawns the side has. *[difference between the side to move and the opponent]*